

System Architecture

Table of Contents

1. [High-level Overview](#)
 2. [Component Diagram](#)
 3. [LangGraph Agent Pipeline](#)
 4. [Data Stores](#)
 5. [Stream Processor Threading Model](#)
 6. [Perception Layer](#)
 7. [Memory & Retrieval Layer](#)
 8. [Dashboard Layer](#)
 9. [Data Structures](#)
 10. [Sequence Diagram — Alert Lifecycle](#)
-

High-level Overview



+-----+ Streamlit Dashboard Live Timeline Ask (RAG) +-----+		
--	--	--

Component Diagram

src/	
+-- perception/	PERCEPTION LAYER
+-- VideoIngestor	Source abstraction (file / RTSP / fallback)
+-- FramePreprocessor	Normalise to 3 parallel views
+-- YOLODetector	Object detection + ByteTrack tracking
+-- VLMCaptioner	Scene description (Bedrock primary / Moondream fallback)
+-- CLIPEmbedder	Semantic 512-dim frame vectors
+-- agent/	AGENT LAYER
+-- graph.py	LangGraph StateGraph assembly
+-- nodes.py	6 node implementations
+-- rule_engine.py	YAML condition evaluator
+-- state.py	AgentState TypedDict
+-- memory/	MEMORY LAYER
+-- SQLiteStore	Relational (frames / objects / events / alerts)
+-- ChromaStore	Vector (CLIP embeddings + metadata)
+-- HybridRetriever	Unified search (semantic + structured)
+-- pipeline/	PIPELINE LAYER
+-- StreamProcessor	Multi-threaded reader + worker
+-- alerts/	ALERT LAYER
+-- AlertDispatcher	Multi-channel (console / email / webhook)
+-- dashboard/	UI LAYER
+-- streamlit_app.py	Live + Timeline + Ask tabs

LangGraph Agent Pipeline

Node responsibility matrix

Node	Reads from state	Writes to state	External I/O
------	------------------	-----------------	--------------

Node	Reads from state	Writes to state	External I/O
perceive	preprocessed, zone	frame_id, ts, detections, caption, embedding	YOLO, CLIP, Bedrock/Moondream
contextualize	ts, zone, detections	track_durations, recent_events, context_summary	SQLite read
rule_check	ts, zone, detections, track_durations, caption	rule_hits, needs_llm	—
llm_judge	rule_hits, context_summary, caption	llm_verdict, llm_alert_msg	Bedrock
alert	rule_hits, llm_verdict, llm_alert_msg, zone, ts	alerts_fired	SQLite write
log	all of the above	logged	SQLite write, ChromaDB write

Routing logic after rule_check

```
def route_after_rule_check(state):
    rule_hits = state.get("rule_hits", [])
    if not rule_hits:
        return "log"                # no threat detected
    if state.get("needs_llm", False):
        return "llm_judge"          # ambiguous -- escalate to LLM
    return "alert"                  # clear rule hit -- fire immediately
```

AgentState TypedDict

```
class AgentState(TypedDict, total=False):
    # Input (set by perceive)
    frame_id: str
    frame_index: int
    ts: datetime
    video_id: str
    zone: str
    preprocessed: PreprocessedFrame
    detections: list[DetectedObject]
    caption: str | None
    embedding: list[float] | None

    # Contextualise output
    track_durations: dict[int, float] # track_id -> seconds in zone
    recent_events: list[dict]
    context_summary: str

    # Rule check output
    rule_hits: list[dict]
    needs_llm: bool

    # LLM judge output
    llm_verdict: str | None # "genuine" | "false_positive"
    llm_alert_msg: str | None

    # Alert output
    alerts_fired: list[dict]

    # Log output
    logged: bool
```

Data Stores

SQLite schema

```

frames
  id          TEXT PRIMARY KEY
  ts          TIMESTAMP
  video_id    TEXT
  frame_index INTEGER
  frame_path  TEXT          (JPEG thumbnail path on disk)
  caption     TEXT          (VLM output)
  telemetry_json TEXT
  zone        TEXT

objects
  id          INTEGER PK AUTOINCREMENT
  frame_id    TEXT -> frames(id)
  class       TEXT          (YOLO class name)
  confidence  REAL
  bbox       TEXT          (JSON [x1,y1,x2,y2])
  track_id    INTEGER

events
  id          TEXT PRIMARY KEY
  start_ts    TIMESTAMP
  end_ts      TIMESTAMP
  type        TEXT          (rule name, e.g. "after_hours_person")
  severity    TEXT          (low / medium / high)
  description TEXT
  frame_ids   TEXT          (JSON list of frame_ids)
  zone        TEXT

alerts
  id          TEXT PRIMARY KEY
  event_id    TEXT -> events(id)
  ts          TIMESTAMP
  channel     TEXT          (console / email / webhook)
  message     TEXT
  acked       INTEGER       (0=unread, 1=acknowledged)

```

Indexes: ts, video_id, zone on frames; class, frame_id, track_id on objects; type, severity, start_ts on events; event_id, acked on alerts.

ChromaDB document structure

```

id:          frame_id (UUID4 string)
embedding: [float * 512]  CLIP ViT-B/32 L2-normalised
metadata:
    frame_index  int
    ts           ISO datetime string
    video_id     str
    zone         str
    caption      str
    class_names  comma-separated YOLO class names

```

Stream Processor Threading Model

```

+-----+
|          StreamProcessor          |
+-----+
|                                     |
|                                     |
ingestor  | +-----+ +-----+      |
.stream() -->| | Reader thread| | Worker thread |      |
|           | |             | |             |      |
|           | | for pkt in  | | while running:|      |
|           | | ingestor:   |-->|  pkt=q.get() |--> result queue
|           | | preprocess | |  agent.invoke|      | (UI polls)
|           | | put(queue) | |  stats.update|      |
|           | +-----+ +-----+      |
|           |                                     |
|           | File mode (drop_on_full=False):      |
|           |   reader.put() blocks when queue is full |
|           |   -> every frame is guaranteed processed |
|           |                                     |
|           | RTSP mode (drop_on_full=True):         |
|           |   oldest frame evicted when queue is full |
|           |   -> low latency, may drop frames      |
+-----+

running = _running OR not queue.empty() OR not results.empty()
(guarantees UI drains all results before marking done)

```

Stats counters

Counter	Description
read	Frames pulled from source
processed	Frames agent.invoke() completed on
dropped	Frames evicted from queue (RTSP mode)

Counter	Description
alerts	Total alerts fired across all frames
drop_pct	dropped / read * 100

Perception Layer

FramePreprocessor — three parallel views

```

FramePacket.image (original BGR ndarray, any resolution)
|
+---> YOLO input:  letterbox to 640x640 BGR uint8
|      (scale + symmetrical pad to preserve aspect ratio)
|      letterbox_meta: {scale, pad_top, pad_left, orig_h, orig_w}
|
+---> VLM input:   resize to 378x378 RGB PIL Image
|      (only populated when run_vlm=True, i.e. on stride)
|
+---> CLIP input:  resize to 224x224 RGB float32 [0,1]

```

VLM stride: `run_vlm = (frame_index % vlm_every == 0)`. Default `vlm_every=5` means 1-in-5 frames gets a caption. On other frames the caption is `None` and caption-based rules do not fire.

VLM async mode (perceive node)

```

Worker frame N:
1. Poll completed VLM futures (non-blocking)  <- captions from past frames
2. Submit VLM job for frame N to thread pool
3. Run YOLO + CLIP on frame N                  <- fast path: ~50ms
4. Return state (caption may be None if not ready yet)

Worker frame N+K (K frames later, ~500ms):
1. Poll completed futures -- frame N's caption arrives
2. caption-based rules fire for frame N+K context

```

Memory & Retrieval Layer

Query routing in HybridRetriever

User query: "show me trucks near the gate on 13 April between 2pm and 4pm"

```

    |
    v
_parse_query_intent()    (LLM call)
returns:
{
    "has_time": true,
    "start": "2026-04-13T14:00:00Z",
    "end":    "2026-04-13T16:00:00Z",
    "class_filter": "truck",
    "zone": "gate",
    "semantic_query": "truck near gate"
}
    |
has_time=True
    |
    v
temporal_search(start, end, class_filter="truck", zone="gate")
SQL: SELECT frames JOIN objects WHERE class=? AND ts BETWEEN ? AND zone=?
returns list[FrameResult] ordered by timestamp
    |
    v
_build_smart_context()
    formats captions + event log into LLM context string
    |
    v
_bedrock_rag_answer(query, context, chat_history)
    Claude Haiku generates answer citing frame timestamps
```

For queries without time context (has_time=False), ChromaDB semantic search is used instead.

Dashboard Layer

Tab structure

```

+-----+
| Drone Security Monitor |
+-----+-----+-----+
| Live | Timeline | Ask |
+-----+-----+-----+

Live tab:
  Sidebar: source type, zone, max frames, storage preset, START/STOP
  Main:    [video frame + bboxes] [metric cards: FPS / alerts / objects]
          [progress bar]          [alert feed (last 100 alerts)]
          [caption strip]

Timeline tab:
  Filters: date range, severity, zone, event type
  Table:   ts | rule | severity badge | zone | frame thumbnail

Ask tab:
  Input:    query text box (multi-turn)
  Response: RAG answer + source frame thumbnails
  History:  last 4 exchanges sent to LLM as conversation context
  Suggest:  3 follow-up question buttons auto-generated after each answer

```

Data Structures

Rule hit (output of RuleEngine.evaluate())

```

{
  "rule_name": str,          # e.g. "after_hours_person"
  "severity":  str,          # "low" | "medium" | "high"
  "message":   str,          # rendered from message_template
  "needs_llm": bool,
  "context":   dict,         # template variables used for rendering
}

```

Alert record (stored in SQLite + returned in alerts_fired)

```

{
    "alert_id": str,          # UUID4
    "rule_name": str,
    "severity": str,
    "message": str,
    "ts": str,               # ISO timestamp
    "zone": str,
}

```

FrameResult (output of HybridRetriever)

```

@dataclass
class FrameResult:
    frame_id: str
    ts: datetime
    zone: str
    caption: str
    class_names: list[str]
    semantic_score: float # cosine similarity (1.0 for SQL-first results)
    frame_index: int
    frame_path: str | None
    telemetry: dict | None

```

Sequence Diagram — Alert Lifecycle

VideoIngestor	StreamProcessor	Agent Graph	SQLite	Dashboard
--FramePacket-->				
	--invoke(state)->			
		--store_event->		
		--store_alert->		
	<--result dict--			
	-----result----->			
			<-get_events--	
			--events list>	
			<-ack_alert---	

System Flow Diagrams

1. End-to-End Frame Processing Flow

```

+-----+
|   Video Source   |
|  MP4 / RTSP /    |
|  Description      |
+-----+-----+
|
|   FramePacket
| {frame_id, ts, video_id, image}
v
+-----+-----+
| FramePreprocessor|   | File mode:      |
|                  |   | reader BLOCKS   |
| yolo 640x640     |   | until worker  |
| vlm 378x378      |   | has space    |
| clip 224x224     |   |                  |
+-----+-----+   | RTSP mode:      |
|                  |   | reader EVICTS |
v                  |   | oldest frame  |
+-----+-----+   +-----+
| StreamProcessor |
| Reader thread   |
| Worker thread   |
| Result queue    |
+-----+-----+
|
| state = {preprocessed, zone}
v
+=====+
| LangGraph Agent Pipeline |
+=====+

Node 1: perceive
+-----+
| YOLODetector.detect()    |
| -> list[DetectedObject] |
| {class, conf, bbox,     |
| track_id}               |
|                          |
| CLIPEmbedder.embed()    |
| -> [float x 512]        |
|                          |
| VLMCaptioner.caption() [async]|
| -> "Two people at gate..." |
+-----+

```

|
v

Node 2: contextualize

```
+-----+
| Track duration accumulator |
|   _track_first_seen dict  |
|   elapsed = now - first_seen |
|   -> {track_id: seconds}   |
|                             |
| SQLite: get_vehicle_counts() |
| SQLite: get_events(last 10min) |
|                             |
| context_summary string      |
|   "Zone: gate | 23:00 | ..." |
+-----+-----+
```

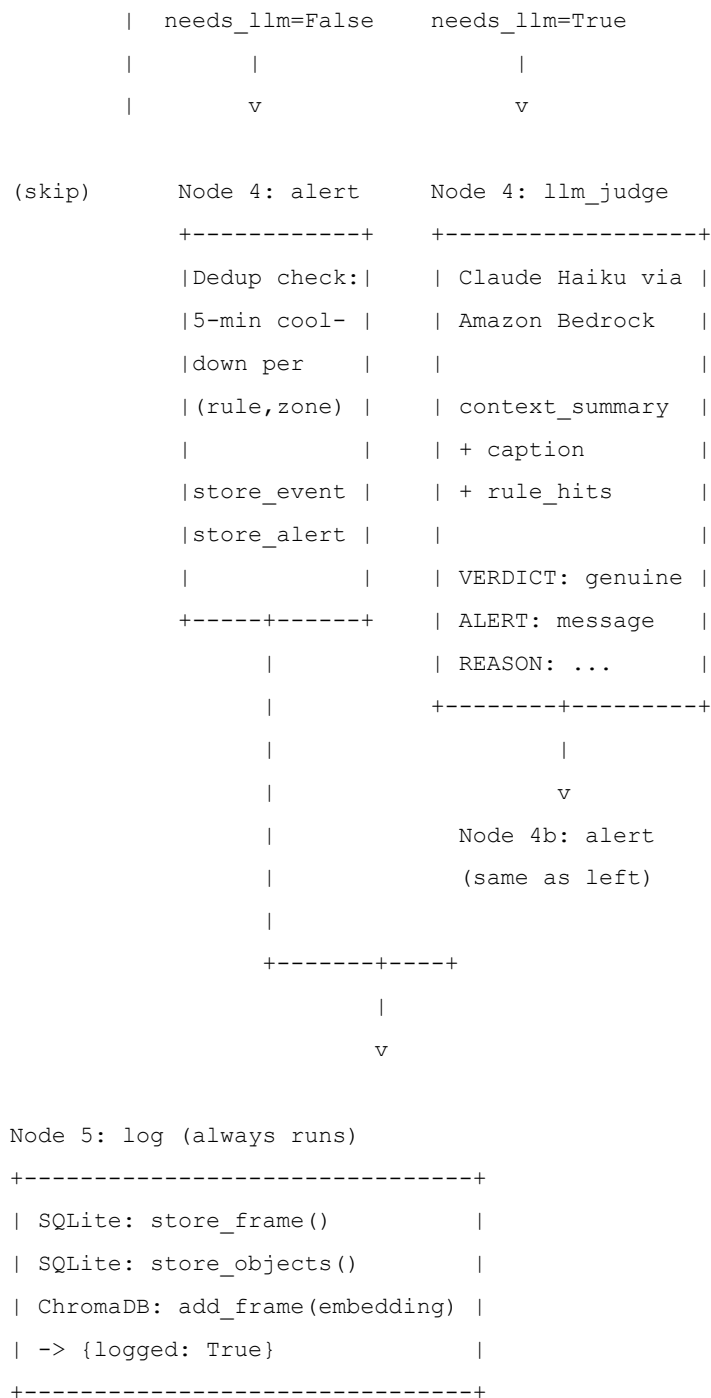
|
v

Node 3: rule_check

```
+-----+
| For each rule in rules.yaml: |
|                               |
|   after_hours_person?        |
|     person detected          |
|     AND 22:00 <= now <= 06:00 |
|     -> HIT (severity=high)    |
|                               |
|   loitering?                 |
|     track_duration >= 60s     |
|     -> HIT (needs_llm=true)   |
|                               |
|   gate_tampering_attempt?    |
|     caption has "tools"+"gate" |
|     -> HIT (needs_llm=true)   |
|                               |
| -> rule_hits[], needs_llm bool |
+-----+-----+
```

|

```
+-----+
|                               |
rule_hits=[]      rule_hits not empty
|                               |
|                               +-----+
|                               |
```



2. Alert Decision Flow

```

rule_hits = RuleEngine.evaluate(ts, zone, detections, track_durations, caption)

|
| empty?
+-----YES-----> log only, no alert
|
NO
|
| any needs_llm=True?
+-----YES-----> llm_judge
|
|
| VERDICT: genuine?
| YES | NO
| | +-> store [LLM: REVIEW] alert
| v
+-----NO-----> alert node
|
| cooldown check:
| same (rule, zone) fired < 5 min ago?
| YES | NO
| | +-> store_event()
| | store_alert()
| | alerts_fired.append()
| |
| suppress (no re-alert)

```

3. Query Routing in the Ask Tab

User types: "show me vehicles near the gate on 13 April"

```

    |
    v
_parse_query_intent() [Bedrock call]
    |
    Returns JSON:
{
    "has_time": true,
    "start": "2026-04-13T00:00:00Z",
    "end": "2026-04-13T23:59:59Z",
    "class_filter": "car",
    "zone": "gate",
    "semantic_query": "vehicles near gate"
}
    |
    has_time == True?
        /           \
        YES           NO
        |             |
        v             v
temporal_search()    chroma.query()
SQLite:              [embed query text]
JOIN objects         [cosine similarity]
WHERE class=car       [top-K frames]
AND ts BETWEEN
AND zone=gate

    |             |
    +-----+-----+
    |
    list[FrameResult]
    |
    _build_smart_context()
        caption snippets
        + event log
        + frame timestamps
    |
    v
    _bedrock_rag_answer()
[Claude Haiku]
[chat_history injected]
    |
    Answer + source frames
    displayed in Ask tab
```

4. Storage Architecture

```

+-----+
|      SQLiteStore      |
| (data/security_memory.db) |
|                          |
| frames ----+          |
| |id|ts|      |          |
| |zone|cap|    |          |
| |path|...|    |          |
|              |          |
| objects <--+         |
| |frame_id| |         | |
| |class|     |         |
| |conf|bbox|  |         |
| |track_id|  |         |
|              v         |
| events      |         | | | |
| |id|type|    |         |
| |severity|   |         |
| |zone|...|   |         |
|              |         |
| alerts <---+        |
| |event_id| |         | |
| |channel|  |         |
| |acked|...| |         |
+-----+
      ^      |
      | writes | reads
      |              v
+-----+-----+-----+
|      Agent Pipeline      |
| log node, alert node,    |
| contextualize node        |
+-----+
      ^
      | ChromaDB (vector store)
      |
+-----+
| collection: security-events |
|                               |
| doc_id: frame_id             |
| embedding: [float x 512]     |
| metadata:                    |
|   ts, zone, caption          |
|   class_names, video_id      |

```

```

+-----+
      ^
      | writes (log node)
      |
      | queries (HybridRetriever)
      v
+-----+
|      HybridRetriever      |
| temporal_search() -> SQL path |
| search()          -> vector path|
+-----+

```

5. Multi-threaded StreamProcessor

```

Main thread (Streamlit)
    |
    | processor.start(ingestor, zone)
    v
+-----+
|          StreamProcessor          |
|                                   |
| +-----+ +-----+ |
| | Reader Thread | | Worker Thread | |
| |               | |               | |
| | for pkt in    | | while True:   | |
| | ingestor:     | |   pkt=q.get    | |
| |               | |   pre=prepr   | |
| | pre=preprocess| |   state={     | |
| |               | |   preproc,   | |
| | RTSP mode:    | |   zone}      | |
| | if q.full():  | |               | |
| |   q.get_nowait | |   result=    | |
| |   dropped++   | |   agent.invoke| |
| |   q.put(pre)   | |               | |
| |               | |   rq.put(res)| |
| | File mode:    | |   stats++    | |
| | q.put(pre)    | |               | |
| | [blocks if    | +-----+ |
| |   full]       | |               |
| +-----+ |
| |               |
| running = _running |
|           OR not q.empty() |
|           OR not rq.empty() |
+-----+
    |
    | processor.get_result(timeout=0.1)
    v
Streamlit UI updates frame + alerts

```

6. VLM Async Processing

Frame N arrives:

```
+-----+
| perceive node                                |
|                                              |
| 1. Poll VLM futures (non-blocking)          |
|   future[N-K].done()? collect caption      |
|                                              |
| 2. Submit VLM job for frame N              |
|   executor.submit(captioner.caption, N)     |
|                                              |
| 3. Run YOLO on frame N      [~30ms]         |
| 4. Run CLIP on frame N     [~20ms]         |
| 5. Return state (caption=None if pending)   |
+-----+
      ... (500ms later) ...
```

Frame N+K arrives:

```
+-----+
| 1. Poll VLM futures                        |
|   future[N].done() -> caption = "... "    |
|   Now caption-based rules can fire        |
|   for context of frame N+K                |
+-----+
```